Table des matières

Labo 02 OLTP — Le langage SQL sur le transactionnel 1
Le terrain, en une minute 1
Ce que ce labo ne refait pas 2
Partie 1 — Lire et trier 3
Partie 2 — Filtrer 5
Partie 3 — Décider et convertir 5
Partie 4 — Le texte 7
Partie 5 — Les dates 8
Partie 6 — Agréger 11
Partie 7 — Ce que le contraste vous a montré 12
Et ensuite ? 13

Labo 02 OLTP — Le langage SQL sur le transactionnel

Laboratoire du Cours 02, voie OLTP · PostgreSQL + AdventureWorks OLTP · Formation SQL avancé

On reprend le laboratoire OLAP sur l'autre terrain : la base transactionnelle `Adventureworks` dans PostgreSQL. Connectez-vous avec pgAdmin, DBeaver ou `psql`. Ce labo ne fait que lire, et il est plus court que son jumeau : il ne garde que les étapes où le terrain change la réponse. Comptez une heure et quart.

Les deux voies — Ce labo est le jumeau de `Labo_02_OLAP_SQL.md`, et il le suppose fait. Les parties et leur numérotation sont les mêmes, les étapes non : celles qui rendraient ici le même verdict que sur l'entrepôt ne sont pas retapées, elles sont listées ci-dessous. Notez chaque écart en chemin, la Partie 7 les rassemble.

Identifiants en minuscules — Le portage PostgreSQL d'AdventureWorks range les tables dans des schémas en minuscules (`production`, `person`, `sales`, `humanresources`) et replie les noms de colonnes de la même façon : `ListPrice` s'écrit `listprice`.

Le terrain, en une minute

© 2026 iAlexMG. Tous droits réservés.

Table	Ce qu'elle contient	Grain	Ce qui servira ici
`production.product`	le catalogue	un produit, 504 lignes	prix, coût, taille, poids, codes de gamme
`person.person`	toutes les personnes : clients, employés, contacts	une personne, 19 972 lignes	prénoms et noms
`person.personphone`	les numéros de téléphone	un numéro, 19 972 lignes	formats de numéro
`humanresources.employee`	le personnel	un employé, 290 lignes	identifiant de connexion, numéro national, dates de naissance et d'embauche
`sales.salesorderheader`	l'en-tête des commandes	une commande, 31 465 lignes	dates, indicateur de vente en ligne
`sales.salesorderdetail`	les lignes de commande	une ligne, 121 317 lignes	quantités

Deux absences par rapport à l'entrepôt : il n'y a ni table de calendrier ni dimension client. Les jours de la semaine se calculent, et les noms des clients vivent dans `person.person`, avec ceux de toutes les autres personnes.

Ce que ce labo ne refait pas

Cinq étapes du labo OLAP ont été relancées ici, sur les tables du transactionnel, et rendent le même verdict : le résultat change, la leçon non. Elles ne sont pas reprises. Lisez cette liste, c'est elle-même un résultat : la plus grande partie du langage ne bouge pas d'un terrain à l'autre.

Étape du labo OLAP	Ce qui a été relancé ici	Verdict
2.1, la priorité de `AND` sur `OR`	63 produits sans parenthèses, 45 avec	même piège, autres chiffres
2.2, `BETWEEN` et sa borne haute	400 commandes de janvier 2024 des deux écritures, aucune commande horodatée ailleurs qu'à minuit	le piège dort ici aussi
2.3, la casse et les accents	`LIKE 'bk-%'` ne trouve rien, `ILIKE 'bk-%'` trouve 97 produits ; deux François avec l'accent, aucun sans	`ILIKE` ignore la casse et respecte l'accent des deux côtés
4.1, le nom complet	19 972 personnes, 11 473 noms complets : 8 499 disparaissent, collés à un second prénom vide	`CONCAT_WS` saute les `NULL` ici comme sur l'entrepôt, l'opérateur de concaténation annule tout

© 2026 iAlexMG. Tous droits réservés.

Étape du labo OLAP	Ce qui a été relancé ici	Verdict
4.3, découper un nom de produit	214 noms à virgule, dont 4 ne finissent pas par la taille	même règle, mêmes exceptions, et `'\'` s'écrit d'une seule barre sur les deux moteurs

Partie 1 — Lire et trier

1.1 — Calculer, nommer, trier

Théorie : `Theorie_02_SQL.md`, section 1.

Objectif : les cinq produits qui dégagent la plus grosse marge.

```
SELECT name,
       listprice,
       standardcost,
       listprice - standardcost AS marge
FROM production.product
ORDER BY marge DESC
LIMIT 5;
```

Observez. Contrairement à l'entrepôt, aucune marge vide n'envahit le haut du palmarès. Les pièces sans prix de vente ont ici un `listprice` à 0, une sentinelle (Cours 01, section 9), et non un `NULL` : leur marge vaut 0 et se range en bas. Les quatre tailles de Mountain-100 Silver arrivent en tête, à égalité sur 1 487,84 $, suivies d'une Mountain-100 Black. Relancez la requête plusieurs fois : rien n'oblige le moteur à garder le même ordre entre les ex aequo. Ajoutez la clé de départage :

```
SELECT name, listprice - standardcost AS marge
FROM production.product
ORDER BY marge DESC, name
LIMIT 5;
```

1.2 — Les valeurs distinctes, et un tri qui trompe

Théorie : `Theorie_02_SQL.md`, section 1.

Objectif : lister les couleurs, puis trier un identifiant numérique rangé en texte.

```
SELECT DISTINCT color
FROM production.product
ORDER BY color;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. Dix lignes : neuf couleurs, puis une ligne vide en dernier. L'entrepôt écrivait « NA », le transactionnel range un vrai `NULL`, et PostgreSQL le place en fin de tri croissant. En tri décroissant, il passe en tête :

```
SELECT name, color
FROM production.product
ORDER BY color DESC
LIMIT 3;
```

Observez. Trois pièces sans couleur, dont Adjustable Race. Le tri de texte qui trompe, lui, est le même qu'à l'étape 1.2 du labo jumeau : le numéro national des employés est rangé en texte, et il se trie caractère par caractère, `10708100` avant `30845`. Ce qui est propre au transactionnel, c'est l'absence : l'entrepôt écrit « NA », il n'a donc rien à ranger en fin de tri.

1.3 — L'alias dans le `WHERE`

Théorie : `Theorie_02_SQL.md`, section 1.

Objectif : voir PostgreSQL refuser ce que DuckDB acceptait.

```
SELECT name, listprice * 1.05 AS prix_avec_tps
FROM production.product
WHERE prix_avec_tps > 3500;
```

Observez. Erreur : « la colonne prix_avec_tps n'existe pas ». Le `WHERE` s'exécute avant le `SELECT`, et PostgreSQL applique l'ordre d'exécution à la lettre. L'écriture qui marche sur les deux moteurs répète l'expression :

```
SELECT name, listprice * 1.05 AS prix_avec_tps
FROM production.product
WHERE listprice * 1.05 > 3500;
```

Observez. 13 produits, les Road-150 et les Mountain-100, comme sur l'entrepôt.

1.4 — L'étoile, sans exception

Théorie : `Theorie_02_SQL.md`, section 1.

Objectif : voir que PostgreSQL n'a pas `EXCLUDE`.

```
SELECT * EXCLUDE (emailpromotion)
FROM person.person
LIMIT 3;
```

Observez. Erreur : « erreur de syntaxe sur ou près de EXCLUDE ». Le mot est une extension de DuckDB. Ici, on garde les colonnes en les nommant :

```
SELECT businessentityid, persontype, firstname, lastname
FROM person.person
```


© 2026 iAlexMG. Tous droits réservés.

```sql
ORDER BY businessentityid
LIMIT 3;
```

Observez. Ken Sánchez, Terri Duffy, Roberto Tamburello, trois employés (`EM`). La liste est plus longue à écrire, mais elle dit exactement ce qui sort, et elle passe sur tous les moteurs.

Partie 2 — Filtrer

2.1 — Le filtre qui trouve

Théorie : `Theorie_02_SQL.md`, sections 2 et 4.

Objectif : compter les produits de la gamme route, codée `R`, et comprendre pourquoi ça marche ici, là où la sous-partie 2.4 du labo jumeau ne trouvait rien.

```sql
SELECT COUNT(*) AS gamme_route
FROM production.product
WHERE productline = 'R';
```

Observez. 100 produits, sans `TRIM`. Sur l'entrepôt, la même requête rendait zéro. Regardez le type :

```sql
SELECT pg_typeof(productline)          AS type,
       COUNT(*) FILTER (WHERE productline = 'R ') AS avec_espace
FROM production.product
GROUP BY pg_typeof(productline);
```

Observez. Le type est `character`, du texte à longueur fixe, et la comparaison avec `'R '` compte aussi 100 produits : un `character` ignore ses espaces de fin. Le portage a gardé le type d'origine, l'entrepôt l'a chargé en `VARCHAR`. Même donnée, autre type, autre résultat. (La clause `FILTER` est un raccourci de PostgreSQL pour compter sous condition ; le Cours 06 la présente.)

Partie 3 — Décider et convertir

3.1 — Étiqueter avec `CASE`

Théorie : `Theorie_02_SQL.md`, sections 3 et 7.

Objectif : répartir les employés selon le genre et le statut familial, en clair.

```sql
SELECT CASE gender WHEN 'F' THEN 'femme' WHEN 'M' THEN 'homme' END              AS genre,
       CASE maritalstatus WHEN 'M' THEN 'marié' WHEN 'S' THEN 'célibataire' END AS etat_civil,
       COUNT(*) AS employes
FROM humanresources.employee
```


© 2026 iAlexMG. Tous droits réservés.

```
GROUP BY 1, 2
ORDER BY 1, 2;
```

Observez. 35 femmes célibataires, 49 femmes mariées, 109 hommes célibataires, 97 hommes mariés. PostgreSQL n'a pas `GROUP BY ALL` : on regroupe par la position des colonnes, `GROUP BY 1, 2`, ou en répétant les deux `CASE`.

Pour un choix à deux branches, pas d'`IFF` ici, le `CASE` fait le travail :

```
SELECT CASE WHEN onlineorderflag THEN 'en ligne' ELSE 'revendeur' END AS canal,
       COUNT(*) AS commandes
FROM sales.salesorderheader
GROUP BY 1;
```

Observez. 27 659 commandes en ligne et 3 806 commandes de revendeurs. `onlineorderflag` est un vrai booléen : la condition s'écrit sans comparaison.

3.2 — Convertir sans tout faire planter

Théorie : `Theorie_02_SQL.md`, section 4.

Objectif : calculer la taille moyenne des produits.

```
SELECT AVG(CAST(size AS INTEGER))
FROM production.product;
```

Observez. Erreur : « syntaxe en entrée invalide pour le type integer : M ». Pas de `TRY_CAST` sur PostgreSQL : on teste la forme de la valeur avant de la convertir.

```
SELECT CASE WHEN size ~ '^[0-9]+$' THEN 'numérique'
            WHEN size IS NULL      THEN 'sans taille'
            ELSE 'lettres'
       END AS genre_de_taille,
       COUNT(*) AS produits
FROM production.product
GROUP BY 1
ORDER BY 1;

SELECT ROUND(AVG(CASE WHEN size ~ '^[0-9]+$' THEN CAST(size AS INTEGER) END), 1) AS taille_moyenne
FROM production.product;
```

Observez. 177 tailles numériques, 34 en lettres, 293 produits sans taille, et une taille moyenne de 49,0. Contrairement à `TRY_CAST`, le `CASE` oblige à écrire la règle de tri entre ce qui se convertit et ce qui ne se convertit pas : plus long, mais rien ne disparaît sans qu'on l'ait décidé.

3.3 — Diviser et arrondir

Théorie : `Theorie_02_SQL.md`, section 4.

© 2026 iAlexMG. Tous droits réservés.

Objectif : la part des commandes passées en ligne.

```
SELECT COUNT(CASE WHEN onlineorderflag THEN 1 END)                    AS en_ligne,
       COUNT(*)                                                       AS commandes,
       COUNT(CASE WHEN onlineorderflag THEN 1 END) / COUNT(*)         AS part,
       ROUND(100.0 * COUNT(CASE WHEN onlineorderflag THEN 1 END) / COUNT(*), 1) AS pourcentage
FROM sales.salesorderheader;
```

Observez. 27 659 commandes en ligne sur 31 465, et une part de 0. La division de deux entiers est entière sur PostgreSQL : 27 659 divisé par 31 465 donne 0 et un reste. La dernière colonne, calculée à partir de `100.0`, donne 87,9 %. Sur l'entrepôt, la même forme de division rendait un décimal. Le motif `COUNT(CASE WHEN … THEN 1 END)` compte les lignes qui remplissent une condition ; le Cours 06 en détaille le mécanisme.

L'arrondi a ses propres exigences de type :

```
SELECT ROUND(AVG(CAST(weight AS FLOAT)), 2)
FROM production.product;
```

Observez. Erreur : « la fonction round(double precision, integer) n'existe pas ». PostgreSQL n'arrondit un `FLOAT` qu'à l'unité. On repasse en `NUMERIC` avant d'arrondir :

```
SELECT ROUND(CAST(AVG(CAST(weight AS FLOAT)) AS NUMERIC), 2) AS poids_moyen
FROM production.product;
```

Observez. 74,07. Dans la vraie vie, `weight` est déjà un `NUMERIC` et `ROUND(AVG(weight), 2)` passe directement ; la conversion en `FLOAT` n'est là que pour faire apparaître la contrainte.

Dernière vérification, la conversion d'unités qui trompait l'entrepôt :

```
SELECT ROUND(MAX(CASE WHEN weightunitmeasurecode = 'G' THEN weight / 453.592
                      ELSE weight END), 2) AS plus_lourd_en_livres,
       MAX(weight)                         AS plus_gros_nombre
FROM production.product;
```

Observez. 30 livres, sans `TRIM` : la colonne d'unité est elle aussi un `character`, et `'G  '` égale `'G'`. Le plus gros nombre de la colonne, 1 050, est une roue mesurée en grammes.

Partie 4 — Le texte

4.1 — Deux formats de téléphone, et l'exception que l'entrepôt n'a pas

Théorie : `Theorie_02_SQL.md`, sections 3 et 5.

Objectif : ramener tous les numéros à la forme 999-999-9999, puis vérifier la règle sur toute la table.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT CASE WHEN phonenumber LIKE '1 (11) %' THEN 'avec indicatif' ELSE 'local' END AS format,
       COUNT(*) AS numeros
FROM person.personphone
GROUP BY 1;
```

Observez. 10 785 numéros locaux et 9 187 précédés de l'indicatif `1 (11)`. La normalisation est la même que sur l'entrepôt :

```sql
SELECT DISTINCT phonenumber,
       CASE WHEN phonenumber LIKE '1 (11) %' THEN REPLACE(SUBSTRING(phonenumber FROM 8), ' ', '-')
            ELSE phonenumber
       END AS telephone
FROM person.personphone
WHERE phonenumber IN ('697-555-0142', '1 (11) 500 555-0190');
```

Observez. `1 (11) 500 555-0190` devient `500-555-0190`. Le `DISTINCT` n'est pas décoratif : ce numéro fictif est partagé par 79 personnes, et sans lui la requête rendrait 80 lignes. `SUBSTRING(phonenumber FROM 8)` est l'écriture de la norme ; `SUBSTR(phonenumber, 8)` marche aussi. Vérifiez maintenant le résultat sur toute la table :

```sql
SELECT phonenumber
FROM person.personphone
WHERE CASE WHEN phonenumber LIKE '1 (11) %' THEN REPLACE(SUBSTRING(phonenumber FROM 8), ' ', '-')
           ELSE phonenumber
      END !~ '^[0-9]{3}-[0-9]{3}-[0-9]{4}$';
```

Observez. Un seul numéro résiste, `55-2555-0100`, qui n'a ni indicatif ni la forme locale. L'entrepôt n'en avait aucun : le transactionnel porte les données brutes, avec leurs exceptions, et une règle de nettoyage se vérifie toujours sur l'ensemble, jamais sur un échantillon.

Partie 5 — Les dates

5.1 — Mesurer un délai

Théorie : `Theorie_02_SQL.md`, section 6.

Objectif : les délais d'expédition, en intervalle puis en jours.

```sql
SELECT shipdate - orderdate AS delai,
       COUNT(*)             AS commandes
FROM sales.salesorderheader
GROUP BY 1
ORDER BY 1;

SELECT CAST(shipdate AS DATE) - CAST(orderdate AS DATE) AS jours,
       COUNT(*)                                         AS commandes
```


© 2026 iAlexMG. Tous droits réservés.

```sql
FROM sales.salesorderheader
GROUP BY 1
ORDER BY 1;
```

Observez. Deux horodatages donnent un intervalle, `7 days` ; deux dates donnent un entier, 7. Et le transactionnel n'est pas tout à fait l'entrepôt : 31 456 commandes partent en 7 jours, mais 9 en 8 jours. PostgreSQL n'a pas de `DATEDIFF` ; la soustraction en tient lieu.

5.2 — Tronquer au mois, nommer le jour

Théorie : `Theorie_02_SQL.md`, section 6.

Objectif : les commandes par mois au premier trimestre 2025, et leur répartition par jour.

```sql
SELECT DATE_TRUNC('month', orderdate) AS mois,
       COUNT(*)                       AS commandes
FROM sales.salesorderheader
WHERE orderdate >= '2025-01-01' AND orderdate < '2025-04-01'
GROUP BY 1
ORDER BY 1;
```

Observez. 2 136 commandes en janvier, 1 857 en février, 2 304 en mars. Regardez maintenant le type rendu selon ce qu'on tronque :

```sql
SELECT pg_typeof(DATE_TRUNC('month', orderdate))                AS sur_un_horodatage,
       pg_typeof(DATE_TRUNC('month', CAST(orderdate AS DATE)))  AS sur_une_date
FROM sales.salesorderheader
LIMIT 1;
```

Observez. Un `timestamp without time zone` d'un côté, un `timestamp with time zone` de l'autre : tronquer une date fait apparaître un fuseau horaire qu'elle n'avait pas. C'est la surprise annoncée par la théorie, et la raison de reconvertir en `DATE` après coup.

```sql
SELECT EXTRACT(ISODOW FROM orderdate) AS jour_iso,
       TO_CHAR(orderdate, 'TMDay')    AS jour,
       COUNT(*)                       AS commandes
FROM sales.salesorderheader
GROUP BY 1, 2
ORDER BY 1;
```

Observez. Du lundi (1) au dimanche (7), en français si votre serveur l'est : le préfixe `TM` de `TMDay` traduit le nom dans la langue du serveur. Le dimanche est le jour le plus chargé, avec 4 875 commandes.

5.3 — L'âge à l'embauche

Théorie : `Theorie_02_SQL.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

Objectif : calculer l'âge des employés au jour de leur embauche.

```sql
SELECT businessentityid, birthdate, hiredate,
       EXTRACT(YEAR FROM hiredate) - EXTRACT(YEAR FROM birthdate) AS age_naif,
       EXTRACT(YEAR FROM AGE(hiredate, birthdate))                AS age_reel
FROM humanresources.employee
ORDER BY businessentityid
LIMIT 3;
```

Observez. Les trois premiers employés ont un an de trop dans la colonne naïve. Le troisième est Roberto Tamburello, que l'entrepôt faisait naître en juin et qui naît ici le 12 novembre 1974, embauché le 11 novembre 2007 : le portage a décalé ses dates, mais il a gardé la veille de l'anniversaire. Combien d'employés sont touchés en tout ?

```sql
SELECT COUNT(*) AS employes,
       COUNT(CASE WHEN EXTRACT(YEAR FROM hiredate) - EXTRACT(YEAR FROM birthdate)
                       <> EXTRACT(YEAR FROM AGE(hiredate, birthdate)) THEN 1 END) AS ages_faux
FROM humanresources.employee;
```

Observez. 184 sur 290. `AGE` rend un intervalle en années, mois et jours, et son nombre d'années est juste d'emblée : aucune correction à écrire, contrairement au `DATEDIFF('year', …)` du labo jumeau.

5.4 — Un horodatage, deux fuseaux, deux jours

Théorie : `Theorie_02_SQL.md`, section 6.

Objectif : le même test que sur l'entrepôt, puis l'aller-retour par l'epoch.

```sql
SELECT CURRENT_SETTING('TimeZone') AS fuseau_de_la_session;
```

Observez. `America/New_York` : le fuseau du serveur, réglé à l'installation, et non celui du poste comme sur DuckDB. Au Québec, les deux ont les mêmes décalages ; un serveur installé ailleurs afficherait les mêmes instants à une autre heure.

```sql
SELECT COUNT(*) AS commandes,
       COUNT(*) FILTER (WHERE CAST(orderdate AT TIME ZONE 'UTC' AT TIME ZONE 'America/Toronto' AS DATE)
                           <> CAST(orderdate AS DATE)) AS changeraient_de_jour
FROM sales.salesorderheader;
```

Observez. 31 465 sur 31 465 : les dates de commande sont des minuits sans fuseau, comme sur l'entrepôt, et lues comme de l'UTC elles tomberaient toutes la veille à Montréal.

```sql
SELECT MIN(orderdate)                                  AS premiere,
       EXTRACT(EPOCH FROM MIN(orderdate))              AS epoch,
       TO_TIMESTAMP(EXTRACT(EPOCH FROM MIN(orderdate))) AS retour
FROM sales.salesorderheader;
```


© 2026 iAlexMG. Tous droits réservés.

Observez. La première commande, `2022-05-30 00:00:00`, devient `1653868800`, puis revient en `2022-05-29 20:00:00-04`. L'aller a compté l'heure de mur comme si elle était en UTC ; le retour rend un instant, affiché à l'heure de l'Est. Un aller-retour qui change de jour, sans une erreur : voilà ce que coûte un epoch calculé sur un `TIMESTAMP`.

Partie 6 — Agréger

6.1 — Ce que compte chaque `COUNT`

Théorie : `Theorie_02_SQL.md`, section 7.

Objectif : vérifier que les deux bases décrivent bien les mêmes ventes.

Les trois `COUNT` de l'étape 6.1 du labo jumeau se comportent ici comme là-bas ; ce qui vaut le détour, c'est le pont entre les deux terrains.

```
SELECT COUNT(*)                    AS lignes,
       COUNT(DISTINCT salesorderid) AS commandes
FROM sales.salesorderdetail;

SELECT COUNT(*)                      AS lignes_en_ligne,
       COUNT(DISTINCT d.salesorderid) AS commandes_en_ligne
FROM sales.salesorderdetail  AS d
JOIN sales.salesorderheader AS h ON h.salesorderid = d.salesorderid
WHERE h.onlineorderflag;
```

Observez. 121 317 lignes pour 31 465 commandes, et, pour les seules ventes en ligne, 60 398 lignes pour 27 659 commandes : exactement les chiffres de l'entrepôt. Les deux bases décrivent les mêmes ventes en ligne, et c'est ce qui rend leur comparaison possible.

6.2 — Les règles du `GROUP BY`

Théorie : `Theorie_02_SQL.md`, section 7.

Objectif : les produits les plus vendus, avec leur couleur.

```
SELECT p.name, p.color, SUM(d.orderqty) AS quantite
FROM sales.salesorderdetail AS d
JOIN production.product     AS p ON p.productid = d.productid
GROUP BY p.name
ORDER BY quantite DESC
LIMIT 3;
```

Observez. Erreur : « la colonne p.color doit apparaître dans la clause GROUP BY ». Regroupez maintenant par la clé primaire du produit :

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT p.name, p.color, SUM(d.orderqty) AS quantite
FROM sales.salesorderdetail AS d
JOIN production.product     AS p ON p.productid = d.productid
GROUP BY p.productid
ORDER BY quantite DESC
LIMIT 3;
```

Observez. La requête passe : AWC Logo Cap (8 311 unités), Water Bottle - 30 oz. (6 815, sans couleur) et Sport-100 Helmet, Blue (6 743). PostgreSQL sait que `name` et `color` dépendent de la clé primaire `productid`, et il les accepte sans les regrouper. DuckDB refuserait la même requête.

Le regroupement accepte aussi l'alias d'une colonne calculée :

```sql
SELECT CASE WHEN listprice = 0   THEN 'sans prix'
            WHEN listprice < 100 THEN 'moins de 100 $'
            ELSE '100 $ et plus'
       END                       AS tranche,
       COUNT(*)                  AS produits,
       ROUND(AVG(listprice), 2)  AS prix_moyen
FROM production.product
GROUP BY tranche
ORDER BY produits DESC;
```

Observez. 214 produits à 100 $ et plus, 200 pièces sans prix, 90 produits à moins de 100 $. L'ordre des `WHEN` compte : la sentinelle 0 est testée avant le seuil de 100 $, sans quoi les 200 pièces sans prix se retrouveraient parmi les produits bon marché.

Partie 7 — Ce que le contraste vous a montré

Théorie : `Theorie_02_SQL.md`, section 8.

Ce qui a changé	Voie OLAP (DuckDB)	Voie OLTP (PostgreSQL)
Un produit sans prix	`ListPrice` à `NULL`, rangé en dernier même en tri décroissant	`listprice` à 0, qui se range en bas
L'alias dans le `WHERE`	accepté	refusé
`SELECT * EXCLUDE (…)`	accepté, avec `REPLACE`	refusé : on nomme les colonnes à garder
Un code de gamme `'R '`	`VARCHAR` : `= 'R'` ne trouve rien sans `TRIM`	`character` : `= 'R'` trouve 100 produits
Une conversion qui peut échouer	`TRY_CAST`, qui change l'échec en `NULL`	`CASE` et expression régulière, qui obligent à écrire la règle

© 2026 iAlexMG. Tous droits réservés.

Ce qui a changé	Voie OLAP (DuckDB)	Voie OLTP (PostgreSQL)
Une part entre deux comptes	décimal d'office	0, sans conversion
`ROUND` sur un `FLOAT`	accepté	refusé, il faut repasser en `NUMERIC`
Un délai entre deux dates	`DATEDIFF('day', …)` ou la soustraction	la soustraction
Tronquer une date au mois	rend un horodatage sans fuseau	rend un horodatage avec fuseau
Un âge	`DATEDIFF('year', …)`, un an de trop pour 111 employés sur 296	`AGE`, juste d'emblée
Le fuseau de la session	celui du poste, `America/Toronto`	celui du serveur, `America/New_York`
Un epoch en millisecondes	`EPOCH_MS`, qui rend une heure de mur en UTC	`TO_TIMESTAMP(n / 1000.0)`, qui rend un instant
Une colonne non agrégée	refusée, sauf `GROUP BY ALL`	acceptée si l'on regroupe par la clé primaire
Les ventes en ligne	60 398 lignes, 27 659 commandes, de 2010 à 2014	les mêmes lignes et commandes, de 2022 à 2025

Trois leçons à emporter. La première : le langage est le même, et presque toutes les requêtes du labo jumeau se transposent mot pour mot. C'est la raison pour laquelle ce labo-ci est court : cinq de ses vingt étapes rendaient ici le même verdict que sur l'entrepôt, et les écarts tiennent à une poignée de fonctions. La deuxième : les écarts les plus dangereux ne lèvent aucune erreur. Le `TRIM` oublié, la division entière et le `DATEDIFF` rendent tous un nombre plausible. La troisième : la même entreprise, décrite par deux bases, ne porte ni les mêmes types, ni les mêmes conventions pour l'absence, ni les mêmes dates. Avant de comparer deux résultats, comparez les deux terrains.

Les exercices du cours et leur corrigé vivent dans `Exercices_corriges_02_SQL.md`. Faites-les après avoir parcouru les deux labos.

Et ensuite ?

Vous savez lire et calculer sur les deux terrains, et vous savez où chacun vous tend ses pièges. Le Cours 03 passe à l'écriture : créer une table, y insérer des lignes, les modifier et les supprimer, et découvrir ce que chaque moteur garantit vraiment quand on déclare une contrainte.

© 2026 iAlexMG. Tous droits réservés.